

CANTATA: A MULTI-HEAD ARCHITECTURE FOR STRUCTURED LONG-FORM MUSIC GENERATION

Omid Aryan

omid@coraltree.ai

Mansour Saffar

mansour@coraltree.ai

ABSTRACT

We present Cantata, a single-stage codec language model for full-length lyrics-to-song generation. Cantata unifies two ideas: the long-context modeling of Llama-style backbones, which sustains coherence across an entire song, and the stacked multi-codebook decoding of neural-codec models, which packs a stream’s parallel residual codebooks into one token sequence—a principle we generalize beyond a single stream. Within one model, it generates the vocal and accompaniment streams jointly, predicting every stream and codebook in parallel at each decoding step. A structured sequence format lays out the whole song in advance: all segment labels (intro, verse, chorus, bridge) and a track description precede generation, and each segment is then produced in turn, conditioned on its lyrics and all previously generated audio. To interleave the two streams’ codebooks into one sequence, we compare several arrangements and adopt accompaniment-first ordering, which yields the lowest training loss. A shared codebook embedding with a lightweight per-stream low-rank residual (in the style of LoRA) makes the design efficiently extensible to more than two streams: a new stem—an instrument, backing vocals, or sound effects—reuses the shared embedding through a small residual rather than a full set of tables. We validate the architecture by training a 3B-parameter model, demonstrating that its many stream-codebook heads optimize stably together and that its loss holds steady across a song’s successive segments.

1. INTRODUCTION

Generative modeling of music has advanced rapidly in recent years, moving from short instrumental clips toward the more ambitious goal of generating complete songs directly from lyrics and text descriptions. Among music generation tasks, full-song lyrics-to-song generation—producing coherent vocals and instrumental accompaniment together, organized into structural sections such as verse, chorus, and bridge—is particularly demanding. Commercial systems such as Suno and Udio have shown that this is achievable at scale, yet their architectures remain proprietary, leaving open how such systems should be designed.

Most open academic systems address this task only partially. Many are constrained to short, single-stream clips, or generate vocals and accompaniment through separate models rather than jointly. Full-song lyrics-to-song generation raises several compounding challenges: 1) *Cross-*

stream coherence: vocals and accompaniment are musically interdependent, and generating them independently loses the bidirectional coupling that characterizes real performance. 2) *Long-range structure*: the model must maintain coherence across structural segments spanning several minutes of audio, far beyond the clip lengths most existing systems target. 3) *Extensibility*: a song may comprise more than two sources—individual instruments, backing vocals, or sound effects—so an architecture should ideally accommodate additional streams, yet most joint designs are hard-coded to exactly two.

Two lines of prior work are most relevant. MusicGen [1] is a single-stage autoregressive model over neural-codec tokens that packs the multiple residual codebooks of a stream into one sequence through codebook interleaving patterns such as delay and parallel; it is, however, limited to a single audio stream, with neither lyrics-driven vocals nor explicit song structure. YuE [2], by contrast, targets full-song lyrics-to-song generation with a Llama-style backbone for long-range coherence, but keeps the prediction decoupled on both axes: its Track-Decoupled Next-Token Prediction (Dual-NTP) alternates between the vocal and accompaniment streams while predicting only their first codebook, leaving the residual codebooks to a separate second-stage model. What is missing is a model that keeps both axes joint—every stream and every residual codebook predicted together at each step—while retaining a language-model backbone and full-song structure.

In this paper, we introduce **Cantata**,¹ a single-stage codec language model built on a Llama backbone [3] that generates all codebook tokens across all audio streams jointly, predicting every stream and codebook in parallel at each decoding step. Cantata conditions on full-song structure—an upfront segment plan, a natural language track description, and per-segment lyrics—and is designed to generate full-length songs with synchronized vocals and accompaniment across multiple structural sections. It is trained end-to-end on multi-GPU AWS infrastructure with DeepSpeed [4] (Section 4).

Beyond joint generation, Cantata’s embedding design is built to scale. A shared codebook embedding specialized per stream by a lightweight low-rank residual, in the style of LoRA [5], lets the architecture incorporate additional audio streams — individual instruments, sound effects, or other stems — at a fraction of the parameter cost of a separate embedding block per stream, a path not available to

¹ Training and inference code, the model architecture, and demo samples are available at <https://github.com/omidmelo/cantata>.

architectures such as YuE’s Dual-NTP that hard-code exactly two streams.

We further study how the two streams should be interleaved. Taking the intra-stream codebook delay of single-stream codec models as given, we formally define five patterns that differ in how the streams are ordered relative to each other, and find that accompaniment-first ordering attains the lowest training loss on real music—indicating it provides the most useful cross-stream conditioning. We validate the overall design by training a 3B-parameter model: all eight stream $\times$ codebook heads optimize stably together, and the loss does not grow across a song’s successive segments. Our results establish the architecture’s feasibility and readiness for scaling rather than benchmarking audio quality, which we leave to future work.

Our main contributions are:

1. **Multi-Head Joint Generation** (Section 3): a single-stage architecture that predicts all codebooks across all streams in parallel at each decoding step, eliminating cascaded inference.
2. **Native Multi-Stream Extensibility** (Section 3.7): a shared codebook embedding with per-stream low-rank residuals (following LoRA) that reuses the codec representation across streams rather than duplicating it, reducing the parameter cost of extending beyond vocals and accompaniment to additional stems.
3. **Interleaving Pattern Study** (Section 3.6): a formal taxonomy of five cross-stream token interleaving patterns—a design axis prior work has fixed ad hoc rather than isolated and compared—together with a comparison identifying the accompaniment-first ordering as the one that attains the lowest training loss for music.

2. RELATED WORK

Audio codec language models. Neural audio codecs based on Residual Vector Quantization (RVQ) [6–8] quantize each short audio frame into a stack of discrete tokens—one per *codebook*, ordered coarse-to-fine across the residual quantization levels—turning a waveform into parallel token sequences that a language model can generate autoregressively. MusicGen [1] is the second point of departure for this work: it formalizes several codebook interleaving strategies—flattening, parallel, delay, and coarse-first patterns—and establishes stacked multi-codebook decoding, in which all of a stream’s residual codebooks are predicted in parallel at each step, as a first-class alternative to cascading. This stacking is confined to a single audio stream; Cantata generalizes it across streams (Section 3). Diffusion-based models such as Stable Audio [9] offer an alternative paradigm operating over continuous latents; we adopt autoregressive modeling for its native support of hard segment boundaries and token-level lyrics alignment.

Long-form lyrics-to-song generation. YuE [2] is the primary point of departure for this work. Its central architectural insight—building on the Llama architecture [3], the design behind modern large language models, rather than a purpose-built music transformer or a cascaded VQ-VAE prior—brings strong long-range sequence-modeling capacity to full-song generation. Cantata adopts the same insight, training this architecture from scratch on audio tokens. YuE is nonetheless a two-stage model: Track-Decoupled Next-Token Prediction (Dual-NTP) alternates first-codebook predictions between the vocal and accompaniment streams before a separate model fills the residual codebooks, while Structural Progressive Conditioning reveals segment labels one at a time as generation proceeds and accumulates previously generated audio as context for each new segment. The broader idea of outlining musical structure before generating audio tokens has also been framed as an explicit chain-of-thought prompting technique for music generation [10]. Cantata follows YuE’s chain-of-thought-style conditioning [11] (Section 3.2) but departs from Dual-NTP through single-stage, all-codebook multi-head generation (a detailed architectural comparison is given in Section 6).

Other lyrics-to-song systems. The remaining systems differ from this design along the same axes—the number of generation stages, the backbone, and how the two streams are combined. Jukebox [12] generates long-form music with a hierarchical VQ-VAE and cascaded transformer priors, conditioning on unaligned lyrics rather than structured segment labels. SongCreator [13] couples two separate vocal and accompaniment decoders with cross-attention, but is two-stage—generating semantic tokens that a separate diffusion model renders to audio—controls acoustics through reference audio prompts rather than text, and is demonstrated only up to 30 seconds rather than full songs. SongGen [14] is a single-stage autoregressive transformer with a mixed mode that generates one combined stream and a dual-track mode that stacks or interleaves the two streams’ tokens. LeVo [15] couples two decoder-only transformers—one modeling combined mixed tokens, the other separate vocal and accompaniment tokens—augmented with DPO-based preference alignment. SongBloom [16] interleaves autoregressive sketching with diffusion refinement, combining the scalability of a language model with the fidelity of diffusion, while SegTune [17] pursues segment-level structure with a non-autoregressive framework that broadcasts local prompts to their corresponding time windows.

Parameter-efficient adaptation. Low-Rank Adaptation (LoRA) [5] shows that a weight matrix can be specialized through a small additive low-rank update rather than a full copy. Cantata’s stream-extensibility challenge has the same shape: the stacked design requires each stream to have its own codebook embedding (Section 3.4), yet duplicating a full table per stream is wasteful. We share one embedding table across streams and supply the per-stream distinction with a low-rank residual in the same style—

applied to the shared embedding rather than to a weight matrix—so each added stream costs a small residual rather than a full table (Section 3.7).

3. ARCHITECTURE

3.1 Overview

Cantata is a single-stage autoregressive codec language model (Figure 1). Given a text conditioning prefix—song structure, description, and lyrics—it generates the discrete audio tokens for all streams and all codebooks of a full song. By a *stream* we mean a separated audio source that is encoded into its own sequence of codec tokens; in this work we consider two streams, vocals and accompaniment. This choice reflects the most common decomposition of a song, but nothing in the architecture is tied to it: the design generalizes to an arbitrary number of streams—for example individual instruments or sound effects—once the interleaving of Section 3.6 is defined over them. By a *segment* we mean a structural section of a song—an intro, verse, chorus, or bridge—which the sequence format represents as a delimited block containing its label, lyrics, and audio.

The model has three components. First, a *unified embedding* maps every token—special tokens, text, and per-stream per-codebook audio tokens—into a single vector space, and sums the multiple codebook embeddings active at each position into one input vector (Sections 3.3 and 3.4). Second, a *Llama backbone* [3]—with rotary position embeddings [18]—processes the resulting sequence, providing the long-range modeling capacity needed for full-song coherence. Third, a bank of *parallel output heads*—one per stream $\times$ codebook—predicts every audio token at each position simultaneously (Section 3.5).

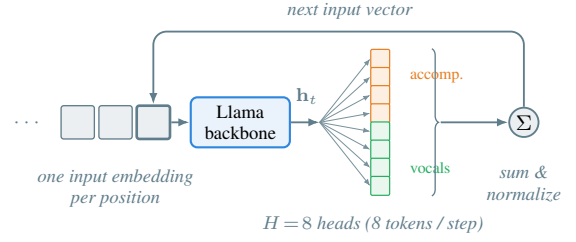
We begin by describing the sequence format and the segment-by-segment generation process (Section 3.2), which gives the document scaffold that the embedding, backbone, and heads operate on; we then detail the vocabulary (Section 3.3), input embedding (Section 3.4), and output heads (Section 3.5), the interleaving of multi-stream tokens (Section 3.6), and finally the shared embedding with per-stream low-rank residuals that lets the design scale to additional streams (Section 3.7).

3.2 Sequence Format and Segment Conditioning

Cantata represents an entire song—its structural plan, lyrics, and audio—as a single flat sequence, or *document*, that the model decodes autoregressively, organized as shown in Figure 2.

The document opens with a global prefix: a start-of-document marker (SOD), the ordered list of *all* segment labels in the song (e.g., *intro*, *verse*, *chorus*, *verse*, *chorus*, *outro*), a natural-language description of the track, and the full lyrics. Each segment then follows as a block delimited by start-of-segment (SOS) and end-of-segment (EOS) markers, containing its own label, its segment-level lyrics, and its audio, interleaved across streams and codebooks

(a) One autoregressive step



(b) One full song

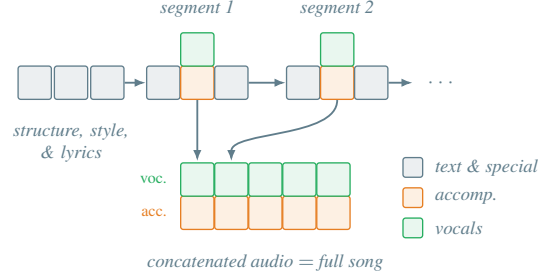


Figure 1. Overview of Cantata at two scales. **(a)** One autoregressive step: the Llama backbone consumes a train of embeddings—one vector per position—and produces a hidden state h_t , from which $H = S \cdot K = 8$ output heads emit every codebook of both streams at once (orange: accompaniment, green: vocals). The step’s eight tokens are embedded and summed back into a single vector (Section 3.4), which becomes the next input—the loop the model repeats to fill one segment. **(b)** One song: a global prefix (structure, style, and lyrics) is followed by segments, each a per-segment prefix, its audio—the accompaniment (orange) and vocal (green) streams stacked—and an end marker. The segments’ audio is concatenated along time into the full song: a two-row grid whose columns are successive segments and whose rows are the vocal (top) and accompaniment (bottom) streams.

by the pattern of Section 3.6. A closing end-of-document marker (EOD) ends the song.

This format follows the chain-of-thought song conditioning of prior work [2, 10, 11], in which a song is generated segment by segment under a fixed vocabulary of segment-label tokens.

Because the segment structure and lyrics are provided as input, generation fills in only the audio, one segment at a time. For each segment, Cantata appends that segment’s given label and lyrics to the running sequence and autoregressively generates its audio, opened by a start-of-audio marker (SOA) and closed when the heads emit their end-of-audio markers (EOA); the rule that decides when the audio is complete is detailed in Section 3.6. The loop then appends the closing EOS and the next segment’s prefix, and the process repeats until EOD. Each segment is therefore generated in the context of the global prefix and every previously generated segment, so its audio is conditioned on the complete musical and textual history that precedes it. Once the full document has been generated, its audio to-

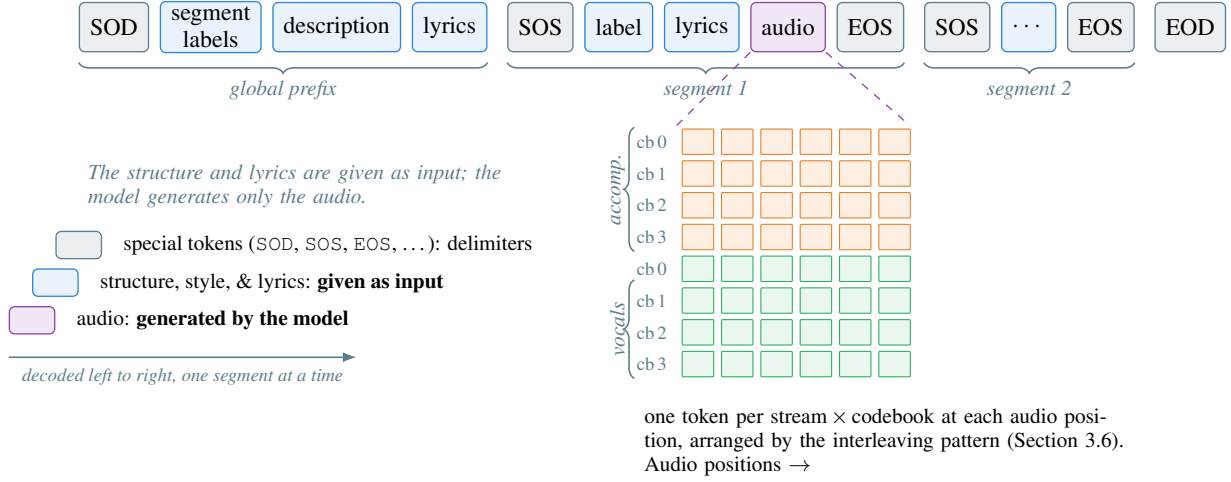


Figure 2. The Cantata sequence format. A song is one flat autoregressive document: a global prefix (all segment labels, a description, and the full lyrics) followed by one block per segment, each holding the segment’s label, lyrics, and audio, and closed by EOD. Each audio region expands (bottom) into one token per stream and codebook at every audio position. At generation time the structure and lyrics are given as input and only the audio is produced, decoded left to right one segment at a time (legend).

kens are de-interleaved and decoded into the final waveform (Section 3.6).

3.3 Unified Vocabulary and Token Registry

Cantata operates over a single flat embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$, where d is the backbone hidden size and the vocabulary V is partitioned into contiguous *blocks*: special tokens (e.g., SOD, SOS, EOS, PAD, segment labels), text tokens (a byte-pair vocabulary for lyrics and descriptions), and audio tokens. A token registry, managed by a *Token-Manager*, records the offset and size of each block and provides the mapping between two distinct index spaces: the *global embedding index space* used on the input side, and a compact *per-head vocabulary space* used on the output side (Section 3.5). Alongside the codec entries, every stream $\times$ codebook is given its own start-of-audio (SOA) and end-of-audio (EOA) marker, opening and closing that head’s run of audio. Both are embeddable inputs; EOA additionally appears in the per-head output space, as the symbol a head predicts to signal that it has finished (Section 3.5).

For the audio tokens, rather than a separate block per stream $\times$ codebook, we allocate one shared embedding table per codebook of a neural codec (e.g., EnCodec [7])— K in total—and reuse it across all streams, specializing it per stream with a lightweight low-rank residual. This shared design, its motivation, and its parameter and extensibility implications are developed in Section 3.7.

Because every token type shares one vocabulary, the model generates audio and consumes text conditioning within a single sequence and a single stage, with no separate text encoder or residual-codebook model. The text block may optionally be initialized from a pretrained Llama embedding and frozen during training, transferring language priors into the lyrics and description representations.

3.4 Embedding Audio Frames

An audio frame is $H = S \cdot K$ tokens—one per codebook for each of the S streams (here $S = 2$ and $K = 4$, so $H = 8$). Since the backbone takes a single vector per position, Cantata embeds these tokens through $\mathbf{E}$ and sums them into one input vector $\mathbf{x}_t$. Two details keep the sum well-behaved: PAD embeddings are zeroed, so inactive heads contribute nothing—some heads hold PAD while the interleaving pattern (Section 3.6) staggers the codebooks in—and, because the resulting number of active tokens n_t can vary, we normalize by $\sqrt{n_t}$,

$$\mathbf{x}_t = \frac{1}{\sqrt{n_t}} \sum_{h=1}^H m_{t,h} \mathbf{E}[i_{t,h}], \quad n_t = \sum_{h=1}^H m_{t,h}, \quad (1)$$

where $i_{t,h}$ is the token id at position t for head h and $m_{t,h} \in \{0, 1\}$ masks PAD. The $1/\sqrt{n_t}$ scaling keeps the variance of $\mathbf{x}_t$ roughly constant as n_t changes. Text and special positions are the degenerate case—a single token, so $\mathbf{x}_t$ is just that token’s embedding, as in a standard language model.

Summing has one consequence for the embedding. When the same codebook k of two streams is active at a position, a single shared table would make the sum $\mathbf{E}_k(a) + \mathbf{E}_k(b)$ invariant to which stream holds a and which holds b , erasing stream identity; the embedding of a codebook token must therefore differ by stream. We meet this requirement without a full table per stream in Section 3.7.

3.5 Multi-Head Output and Loss

From each backbone hidden state $\mathbf{h}_t \in \mathbb{R}^d$, Cantata predicts every audio token in parallel through $H = S \cdot K$ output heads, one per stream $\times$ codebook. Each head outputs a distribution over the compact per-head vocabulary—the codec’s codebook entries plus an end-of-audio (EOA)

symbol—rather than the full global vocabulary, keeping the heads small. Concretely, if a codebook has N entries (indices $0, \dots, N-1$), EOA is the extra class at index N : a head signals that its stream has finished by predicting EOA in place at an audio position. Unlike the document delimiters `SOS` and `EOS`, which each occupy their own position in the sequence, EOA therefore consumes no separate position—it is a value a head can emit at an audio position, not a token of its own. All heads read the single hidden state $\mathbf{h}_t$ and are computed simultaneously, so one forward step yields a complete audio frame: all K codebooks for all S streams at once. Each head has its own weights, so the two streams produce distinct predictions for the same codebook despite reading a common hidden state.

Training uses a next-token objective averaged over heads. Writing $\ell_{t,h}$ for the cross-entropy of head h at position t (predicting position $t+1$ from t) and $\mathcal{T}_h$ for the set of valid target positions of head h , the loss is $\mathcal{L} = \frac{1}{H} \sum_{h=1}^H \frac{1}{|\mathcal{T}_h|} \sum_{t \in \mathcal{T}_h} \ell_{t,h}$. The loss is computed only over audio tokens and the EOA symbol: structural positions determined by the interleaving pattern (PAD and the start-of-audio marker SOA) and all text conditioning tokens are assigned the ignore label and excluded from $\mathcal{T}_h$.

This multi-head design keeps the architecture open to additional streams: a new stream adds its own K heads and reuses the backbone unchanged, with no second stage introduced; the corresponding input-side cost is addressed by the shared embedding of Section 3.7. (The interleaving patterns of Section 3.6 are, in their current form, defined for two streams; generalizing the pattern taxonomy to more streams is future work.) The inference and training efficiency consequences of this single-stage, all-parallel design are discussed in Section 6.

3.6 Token Interleaving Patterns

Within a segment’s audio region, the model must arrange the $S \cdot K$ audio tokens of each frame— K codebooks for each of S streams—into the single autoregressive sequence consumed by the backbone. This arrangement is not incidental: it determines the causal dependency structure between codebooks and between streams, since a token placed at a later position may condition on all tokens at earlier positions but not the reverse.

A frame’s codebooks cannot simply be stacked at one position and emitted together: that arrangement models them as conditionally independent given the history, an assumption residual quantization violates by construction, since each codebook encodes the residual left by the previous one [1]. We therefore adopt the intra-stream codebook *delay* of single-stream codec models throughout—staggering codebook k so that finer codebooks may condition on coarser ones of the same frame—and focus on the question our two-stream setting adds: how to order the two streams relative to each other. We treat this cross-stream arrangement as an explicit, configurable design choice and study a family of five patterns, all of which share the codebook delay.

Notation. We index heads by (s, k) for stream

$s \in \{a, v\}$ (accompaniment and vocals) and codebook $k \in \{0, \dots, K-1\}$, ordering them as $[a_0, \dots, a_{K-1}, v_0, \dots, v_{K-1}]$. A pattern is defined by a per-head *injection offset* $\tau(s, k)$: within the audio region, letting $t = 0, 1, 2, \dots$ index audio steps, head (s, k) emits PAD while $t < \tau(s, k)$, emits its start-of-audio marker SOA at $t = \tau(s, k)$, and emits the $(t - \tau(s, k))$ -th codec token thereafter. The offset function alone fully specifies a pattern; Figure 3 shows the five we consider, all sharing the intra-stream codebook delay (τ increasing with k) and differing only in how the two streams are offset relative to each other.

Interpretation. The five patterns span a spectrum of cross-stream coupling, all built on the shared codebook delay. *Diagonal delay* staggers codebook k by k steps within each stream—the single-stream delay pattern applied to both streams symmetrically—but adds no offset between the streams: the two advance in lockstep, so neither is placed ahead of the other to condition on. It is the natural baseline against which the remaining four introduce a directional cross-stream dependency. *Accompaniment-first* and *vocals-first* additionally offset one stream by a single step relative to the other, letting the trailing stream condition on the leading stream’s corresponding tokens; these encode a directional cross-stream dependency (for example, vocals conditioned on accompaniment). The *sequential* patterns take this to its extreme, emitting one stream in full before the other begins, the strongest one-directional coupling but also the longest delay.

Stopping. At generation time, because the heads are offset, they reach the end of the audio region at different steps, so the audio is not complete the moment the first head emits its EOA. Cantata therefore stops with a *pattern-spread* rule: after the first EOA, it continues for exactly as many steps as separate the last head to finish from the first, letting every head emit its own EOA and holding already-finished heads at PAD in the meantime. This spread is fixed by the pattern— $K-1$ steps for diagonal delay, K for the single-step cross-stream patterns, and $2K-1$ for the sequential patterns—so no audio is truncated. (Simply halting on the first EOA is possible but drops the still-unfinished heads’ remaining frames, so we do not use it.)

Which pattern to use. Because all five patterns predict the same audio tokens and differ only in the dependency ordering imposed on them, a lower training loss reflects a more informative conditioning structure rather than an easier objective. Comparing the five under identical settings (Figure 4), *accompaniment-first* attains the lowest training loss, indicating that conditioning the vocals on the corresponding accompaniment tokens provides the most useful cross-stream context. We take this as evidence that accompaniment—the harmonically richer stream—is best placed to lead; Section 6 returns to why.

From tokens to audio. Recovering a waveform reverses this layout. For each segment we de-interleave the generated audio tokens—undoing the pattern so that each stream

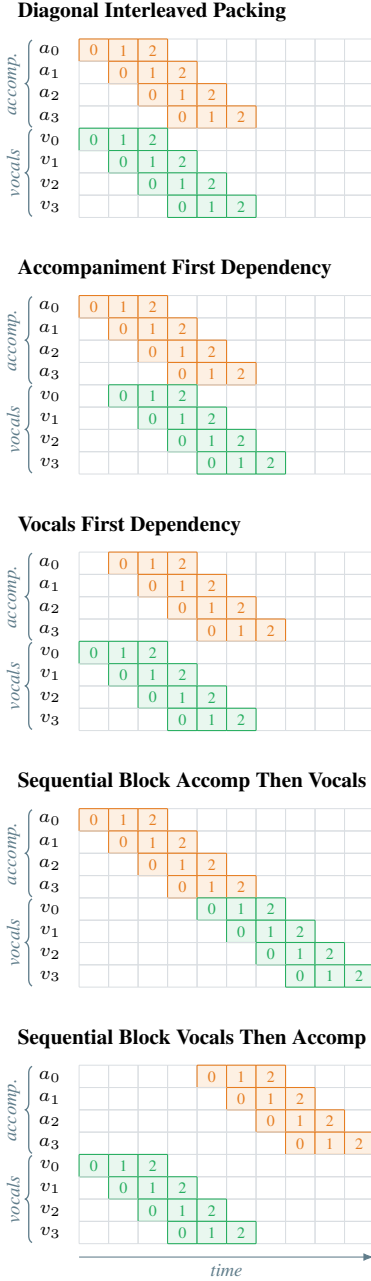


Figure 3. The five cross-stream interleaving patterns, drawn as head $\times$ timestep grids over a common number of steps: rows are the eight heads, grouped into the four accompaniment codebooks (a_0 – a_3) and the four vocals codebooks (v_0 – v_3), with time running left to right. A cell is filled when that head emits a codec token; its colour marks the stream and the digit inside is the frame index, and blank cells are PAD. All five share the within-stream delay (leading stream $\tau = k$) and differ in the cross-stream offset: accompaniment-first delays the vocals by one step ($\tau(v, k) = k+1$), vocals-first delays the accompaniment ($\tau(a, k) = k+1$), and the sequential patterns delay the trailing stream by a full block ($\tau = K+k$).

is again a rectangular block of K codebooks over time—and concatenate the per-segment blocks along time into one token sequence per stream. Each stream is then de-

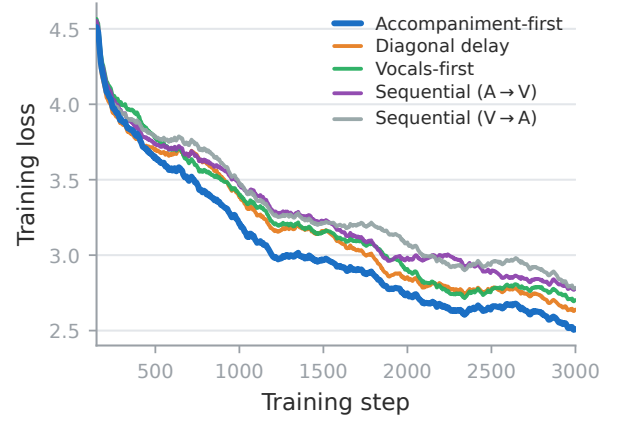


Figure 4. Training loss for the five interleaving patterns under identical settings. Accompaniment-first converges to the lowest loss.

coded independently by the EnCodec decoder [7], and the two resulting waveforms are combined into the full song.

3.7 Shared Codebook Embedding with Per-Stream Low-Rank Residuals

Section 3.4 established that our stacked design *requires* the embedding of a codebook token to differ by stream: because same-codebook tokens from different streams are summed at a shared position, a single table would make the input invariant to which stream holds which token. The question this section answers is how to supply that per-stream distinction.

The naive way is to allocate a separate embedding table for every stream $\times$ codebook, so that vocals and accompaniment learn independent representations of codebook k from scratch. This meets the requirement but is wasteful, and for a reason specific to how a codec represents audio: under the assumption we adopt throughout—that a single codec encodes every stream—codebook k indexes the *same* vocabulary regardless of stream, so token x of codebook k denotes the same acoustic unit whether it appears in vocals or accompaniment. Most of what a per-stream table would learn is therefore common to all streams; only a comparatively small residual is stream-specific.

We exploit this by factoring the two apart. We learn a single shared embedding table $\mathbf{E}_k \in \mathbb{R}^{V \times d}$ per codebook, reused across all streams to carry the common codec-level representation, and add a lightweight *low-rank residual* that captures the per-stream variation. For stream s and codebook k , the input embedding is

$$\mathbf{E}_{s,k}(x) = \mathbf{E}_k(x) + \mathbf{B}_s \mathbf{A}_s \mathbf{E}_k(x), \quad (2)$$

where $\mathbf{A}_s \in \mathbb{R}^{r \times d}$ and $\mathbf{B}_s \in \mathbb{R}^{d \times r}$ are rank- r factors ($r \ll d$) indexed by stream only, so a single residual per stream is reused across all of that stream’s codebooks. The parameterization is that of LoRA [5]—a low-rank product $\mathbf{B}_s \mathbf{A}_s$ —but applied to the shared embedding vector rather than to a frozen weight matrix, so it acts as an additive correction to the codec representation. The output heads are

unaffected: each stream $\times$ codebook keeps its own readout as described in Section 3.5.

This factorization buys two things over the naive separate-block baseline. *First, more efficient training.* With S streams, K codebooks, vocabulary V , and hidden size d , the baseline spends $SKVd$ parameters on audio embeddings, all of them trained. The shared design trains KVd for the shared tables plus only $O(Srd)$ for the residuals ($r \ll V$), so each added stream contributes a small residual rather than a full duplicate of the largest token-specific component; fewer trained parameters carrying the per-stream distinction means less to fit and a smaller optimization footprint per stream. *Second, cheaper and better-initialized extensibility.* Because the shared tables already encode the codec-level representation learned across every stream seen in training, a stream added later—a new stem, say—does not start from scratch: it inherits that representation immediately and needs to learn only its low-rank residual and its K output heads. Adding a stream is thus both inexpensive and warm-started by transfer from the streams already modeled, rather than requiring a fresh embedding table to be trained from nothing.

One extension we leave to future work is relaxing the single-codec assumption—were streams encoded by different codecs, each would retain its own codebook tables and sharing would apply only among streams whose codecs coincide.

4. TRAINING SETUP

4.1 Training Data

Cantata is trained on a single corpus that combines music and speech, both cast into the unified segment-structured token sequence of Section 3.2 so that the model consumes every example in one format. The music portion comprises roughly 250K songs, all with English lyrics when vocals are present, drawn from Creative Commons and other publicly available online sources. The speech portion is LibriHeavy [19], a 50,000-hour English read-speech corpus with reliable transcripts. Speech is included to strengthen lyric-following: music collected in the wild has imperfect lyric-to-audio alignment, whereas transcribed speech provides an unambiguous mapping from text to vocalization, an auxiliary signal also found important by YuE [2]. Each example is tagged with a MUSIC or SPEECH token, and we sample music for roughly 70% of training batches and speech for the remaining 30%.

Each music track is derived into the sequence of Section 3.2 through a multi-stage pipeline. Demucs [20] first separates the track into vocal and accompaniment stems, which become the two audio streams; each stem is then tokenized independently by EnCodec [7] at 32 kHz into four codebooks. In parallel, Faster-Whisper [21] transcribes the vocal stem to obtain sung lyrics with word-level timestamps, and all-in-one [22] detects structural boundaries and their labels (intro, verse, chorus, and so on). Reference lyrics are aligned to the transcribed timestamps by longest-common-subsequence matching to yield clean per-

segment lyrics. A natural-language description of the track is produced in two stages: audio taggers—PANNs [23] and a CLAP-based [24] tagger we train on our own data and label set—first extract structured attributes (genre, instrumentation, mood, tempo, and the like), and a language model then renders these tags into a short free-form description suitable as a conditioning prefix. We adopt this tag-then-describe pipeline because we found end-to-end music-captioning models insufficiently accurate and unable to surface all the attributes we condition on. These signals are assembled into the global prefix and per-segment blocks of Section 3.2; instrumental tracks, which have no vocals, use a silent vocal stream, and, symmetrically, speech from LibriHeavy, which has no accompaniment, uses a silent accompaniment stream, with the provided transcript in place of lyrics. We hold out 10% of the corpus as an evaluation set, disjoint from the training data, on which we periodically measure the evaluation loss reported in Section 5.

4.2 Model and Optimization

The backbone is a Llama transformer [3, 25] with hidden size 2560, 24 layers, and 20 attention heads (multi-head attention), using rotary position embeddings [18] and a context length of 8200 tokens. Audio is represented with EnCodec (32 kHz, four codebooks) over the two streams, accompaniment and vocals; text is tokenized with a Llama-2 tokenizer over a 32K vocabulary. The model has approximately 3B parameters.

We train on a single AWS P5.48xlarge instance with eight H100 80 GB GPUs using DeepSpeed ZeRO [4], with an effective batch size of 768 (per-device batch times gradient accumulation of 12) for approximately 30K steps. We use the AdamW optimizer (learning rate 3×10^{-4} , weight decay 0.1, $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$) with a constant learning-rate schedule preceded by a 2500-step linear warmup. Training uses bfloat16 mixed precision with TF32 enabled.

5. EXPERIMENTS

We report the training behavior of the full-scale 3B model of Section 4. Training and evaluation loss both fall smoothly and monotonically, with no spikes or divergence, and the evaluation loss tracks the training loss throughout, showing no sign of overfitting. The analyses below ask how that converged loss is *distributed*—first across the model’s heads, then across a song’s successive segments. Per-head values come from the model’s logged training metrics; per-segment values from a separate teacher-forced evaluation pass over held-out songs.

Multi-head stability: all eight heads optimize together.

The central risk of a joint objective over many heads is uneven optimization—some heads dominating while others stall or collapse. Because the model exposes one loss per stream $\times$ codebook, we can check this directly (Figure 5). All eight heads fall from initialization and converge, none diverging or stalling. The multi-head objective therefore

optimizes stably across every stream and codebook at 3B scale, which is the architecture’s central feasibility claim.

Stream asymmetry: accompaniment carries more information. The same view exposes a pronounced gap between the streams. The vocal heads settle low and tightly clustered, whereas the accompaniment heads sit between roughly $1.8\times$ and $2.3\times$ higher, the gap widening as the codebooks grow finer. The cause lies in the streams themselves rather than in any interleaving choice: the vocal signal is monophonic and tightly constrained by the lyric conditioning, so its next token is highly predictable, whereas the accompaniment is polyphonic, carries no direct textual conditioning, and is spectrally far more diverse, giving it substantially higher per-token entropy. This supports the accompaniment-first ordering of Section 3.6 from a second direction: the higher-information stream is the one whose early prediction most constrains the other.

Loss also rises with codebook index within each stream. This follows the residual structure of the codec: the coarsest codebook is the easiest to predict, and each successive residual carries progressively finer, more noise-like detail.

Long-form stability: loss does not grow across a song. A long-form model is only useful if its quality holds up over the length of a song, so we measure the loss at each segment position separately (Figure 6). Segments two through five sit within roughly 0.16 of one another, showing no upward trend with position: the model predicts later segments, with their longer conditioning history, as accurately as earlier ones. The first segment is easiest by a visible margin, which we attribute to its content rather than to the model—a song’s opening segment is typically an intro, sparser and more formulaic than the verses and choruses that follow.

Two details of the measurement are worth noting. The loss uses the same head-averaged formulation as the training objective, windowed to one segment’s token range, so these values are commensurate with the per-head losses above. The population is restricted to songs with at least five non-truncated segments, scoring only their first five, so every position is measured over the same songs rather than a subset that shrinks as songs get shorter.

Because the loss is computed with ground-truth context, this isolates one of the two ways long-form modeling can fail. It shows that the model’s use of context does not degrade with position, ruling out context-length degradation—the failure mode the long-context backbone exists to prevent. It does not speak to error accumulation under free-running generation, where the model conditions on its own output; we leave that to future work.

6. CONCLUSION

We introduced Cantata, a single-stage, multi-head codec language model that jointly generates the vocal and accompaniment streams of a full song, conditioned on structured segment-level information. The current model and results should be read as a technical validation of this architecture rather than a production-ready generator: they

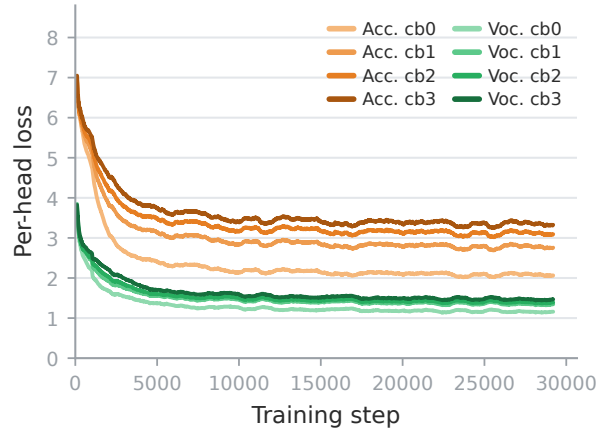


Figure 5. Per-head training loss for all eight stream×codebook heads on a shared axis over the whole run (running mean over 200 logged points). All eight fall from initialization and converge, none diverging or stalling. Accompaniment (orange) sits well above vocals (green) throughout, and within each stream loss increases with codebook index.

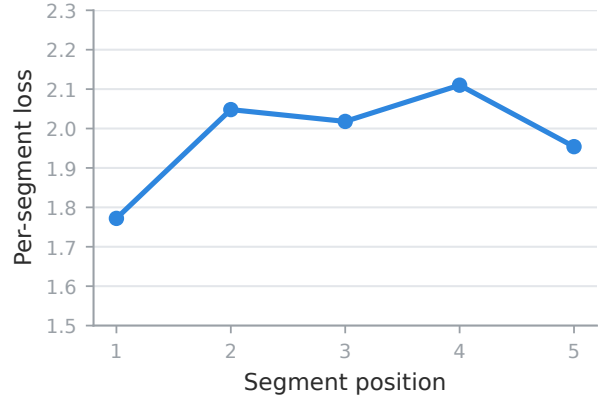


Figure 6. Per-segment loss at the final checkpoint by segment position, over the evaluation set restricted to songs with at least five segments (values 1.77, 2.05, 2.02, 2.11, 1.95; note the narrow vertical range). Loss does not grow with position: the first segment—typically an intro—is easiest, and the remaining four differ by only about 0.16 with no upward trend.

establish that the design is feasible, that it optimizes stably across all streams at the 3B-parameter scale, and that it is ready for continued scaling. The trained model produces coherent song excerpts in favorable cases, but we make no claim of production-quality generation and leave a systematic perceptual evaluation to future work.

Two of our empirical observations reinforce one another. The per-head losses reveal a pronounced asymmetry between the streams: the accompaniment is markedly harder to predict than the lyric-constrained vocals, reflecting its polyphonic, higher-entropy, and unconditioned nature. This clarifies why accompaniment-first interleaving attains the lowest loss among the five patterns (Sec-

tion 3.6): the harmonically richer stream carries more information, so predicting it first supplies the vocal stream with the most useful conditioning context. The most effective ordering is the one that lets the higher-information stream ground the lower-information one.

A third observation speaks to the long-form goal directly. Measured per segment position over the evaluation set, the loss is essentially flat from the second segment onward, with no upward trend as the conditioning history lengthens—the behavior the long-context backbone is meant to deliver.

Inference and training efficiency. Cantata’s single-stage, all-parallel design is cheaper to run and to train than two-stage, sequential-token approaches such as YuE’s Dual-NTP [2]. Consider generating a passage of N audio frames. *Fewer decoding steps:* Cantata predicts all codebooks of both streams at once, so it needs about N sequential steps within a single model, whereas Dual-NTP predicts one codebook at a time and alternates between the streams—about $2N$ steps just for the first codebook of each frame, after which a *separate* model fills in the rest; Cantata thus reaches the same audio in more than $2\times$ fewer autoregressive steps. *Shorter context:* Cantata stacks all codebooks at one position and represents the audio in about N sequence positions, versus roughly $2N$ for Dual-NTP’s first-codebook stage, leaving more of the context window for musical history. *Simpler pipeline:* one model trained end to end, rather than two separately trained stages.

Limitations. Our evaluation establishes training behavior and the interleaving comparison but does not measure audio quality: we report no Fréchet Audio Distance or listening-test results, and our claims are correspondingly architectural. We train and evaluate two streams—vocals and accompaniment—and a single codec family (EnCodec); extensibility to further streams and codecs is a property of the design rather than an empirically demonstrated result. The pattern comparison likewise rests on a single metric, training loss.

Future work. Natural next steps include validating the architecture on more than two streams, such as individual instruments or sound effects; extending the shared-embedding-with-residuals scheme to the output heads and to heterogeneous per-stream codecs; adopting higher-fidelity codecs such as DAC [8]; handling longer contexts as songs grow in length; and carrying out a full perceptual evaluation. We believe the single-stage, multi-head formulation—joint over all streams and codebooks, extensible by construction, and validated here at scale—is a solid foundation for these directions.

7. ACKNOWLEDGMENTS

This work was supported by the NVIDIA Inception Program and AWS Activate. We thank both programs for providing compute credits and resources that made the large-scale training of Cantata possible.

8. REFERENCES

- [1] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, “Simple and controllable music generation,” in *Proc. of Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [2] R. Yuan *et al.*, “YuE: Scaling open foundation models for long-form music generation,” in *arXiv preprint arXiv:2503.08638*, 2025.
- [3] H. Touvron *et al.*, “LLaMA: Open and efficient foundation language models,” in *arXiv preprint arXiv:2302.13971*, 2023.
- [4] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory optimizations toward training trillion parameter models,” in *Proc. of the Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. of the Int. Conf. on Learning Representations (ICLR)*, 2022.
- [6] N. Zeghidour, A. Luebs, A. Omran, J.-M. Skoglund, and M. Tagliasacchi, “SoundStream: An end-to-end neural audio codec,” in *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021.
- [7] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression,” in *Proc. of the Int. Conf. on Learning Representations (ICLR)*, 2023.
- [8] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, “High-fidelity audio compression with improved RVQGAN,” in *Proc. of Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [9] Z. Evans, C. Carr, J. Taylor, S. H. Hawley, and J. Pons, “Stable audio: Fast timing-conditioned latent audio diffusion,” in *arXiv preprint arXiv:2402.04825*, 2024.
- [10] M. W. Y. Lam, Y. Xing, W. You, J. Wu, Z. Yin, F. Jiang, H. Liu, F. Liu, X. Li, W.-T. Lu, H. Chen, T. Feng, T. Zhao, C.-H. Liu, X. Song, Y. Li, and Y. Zhou, “Analyzable chain-of-musical-thought prompting for high-fidelity music generation,” in *arXiv preprint arXiv:2503.19611*, 2025.
- [11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. of Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [12] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, “Jukebox: A generative model for music,” in *arXiv preprint arXiv:2005.00341*, 2020.
- [13] X. Lei *et al.*, “SongCreator: Lyrics-based universal song generation,” in *arXiv preprint arXiv:2409.06029*, 2024.
- [14] Z. Liu, S. Ding, Z. Zhang, X. Dong, P. Zhang, Y. Zang, Y. Cao, D. Lin, and J. Wang, “SongGen: A single stage auto-regressive transformer for text-to-song generation,” in *arXiv preprint arXiv:2502.13128*, 2025.
- [15] S. Lei, Y. Xu, Z. Lin, H. Zhang, W. Tan, H. Chen, J. Yu, Y. Zhang, C. Yang, H. Zhu, S. Wang, Z. Wu, and D. Yu, “LeVo: High-quality song generation

- with multi-preference alignment,” in *arXiv preprint arXiv:2506.07520*, 2025.
- [16] C. Yang, S. Wang, H. Chen, W. Tan, J. Yu, and H. Li, “SongBloom: Coherent song generation via interleaved autoregressive sketching and diffusion refinement,” in *arXiv preprint arXiv:2506.07634*, 2025.
 - [17] P. Cai, J. Wang, H. Zheng, X. Li, Z. Ji, T. Ma, Z. Liu, C. Zhang, and P. Wan, “SegTune: Structured and fine-grained control for song generation,” in *arXiv preprint arXiv:2510.18416*, 2025.
 - [18] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021.
 - [19] W. Kang, X. Yang, Z. Yao, F. Kuang, Y. Yang, L. Guo, L. Lin, and D. Povey, “Libriheavy: A 50,000 hours ASR corpus with punctuation casing and context,” in *Proc. of the IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.
 - [20] A. Défossez, “Hybrid spectrogram and waveform source separation,” in *Proc. of the ISMIR Workshop on Music Source Separation*, 2021.
 - [21] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. of the Int. Conf. on Machine Learning (ICML)*, 2023.
 - [22] M. Kim and J. Nam, “All-in-one metrical and functional structure analysis with neighborhood attentions on demixed audio,” in *arXiv preprint arXiv:2307.16425*, 2023.
 - [23] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “PANNS: Large-scale pretrained audio neural networks for audio pattern recognition,” in *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2020.
 - [24] Y. Wu, K. Chen, T. Zhang, Y. Hui, M. Nezhurina, T. Berg-Kirkpatrick, and S. Dubnov, “Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation,” in *Proc. of the IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.
 - [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. of Advances in Neural Information Processing Systems (NeurIPS)*, 2017.